SubgoalCalc: Certified Decomposition for Lean Theorem-Proving Agents

Anonymous Authors

May 13, 2026

Abstract

Subgoals now drive theorem-proving agents across extraction, search, scheduling, and reuse. We present SUBGOALCALC, a certified decomposition layer for Lean 4 that turns subgoals into proof obligations carrying witnesses, dependencies, and replay semantics. Its central object is a certified decomposition: a root goal, residual obligations, an acyclic dependency relation, and a Lean-checked assembly witness reconstructing the root theorem from residual proofs. The layer provides four exact operators: normalization, duplicate collapse, overlay, and refinement; together with dependency-aware schedule selection and replay-backed lemma lifting.

Across exact suites, the operator basis reduces 1,000 certified flat decompositions to four structural cores; a dependency suite contributes 750 certified DAGs with depth up to 27; and 1,500 certified transforms change execution mode exactly at crossings of the dependency-edge boundary. In HILBERT, kernel-faithful extraction raises end-to-end success from 0/6 to 6/6 over textual extraction and from 4/6 to 6/6 over a retry baseline, with 12/12 helper replays, 6/6 dependent-helper replays, and zero extraction-related recovery calls. On DEPBENCH, certified dependencies select staged execution exactly on the four dependency-bearing cases, raising contract satisfaction from 2/6 to 6/6 and reducing schedule cost from 54 to 14. Replay-backed helper lemmas compress chain plans, transfer across held-out HILBERT targets, and flip one DEPBENCH failure to success. The same control plane closes a white-box AESOP slice from 0/3 to 3/3 and produces exact certified decompositions for all 4,937 FORMALML examples, localizing the remaining replay boundary to provisioning, serializer effects, and source-aligned repair.

1 Introduction

Subgoal structure now appears throughout formal theorem proving. FormalML isolates subgoal completion as a distinct benchmark task in Lean 4 [Y]; Zhao et al. and SubgoalXL use subgoal-based proofs to improve in-context proving and expert learning in Isabelle [Z, Z]; HILBERT uses recursive decomposition to connect informal reasoning with formal proof construction [V]; DeepSeek-Prover-V2 uses subgoal decomposition to build synthetic training data and stabilize

reinforcement learning [R]; Bourbaki treats self-generated subgoals as dynamic search targets [Z]; and MathlibLemma shows that many missing intermediate facts are natural candidates for library growth [L]. Across these systems, subgoals organize prover work by structuring generation, search, training, and reuse around intermediate obligations.

The corresponding representations usually materialize subgoals as proof sketches, helper-theorem strings, or search states. These forms can drive generation and search, but they do not by themselves record the operational structure needed to audit and reuse a lifted obligation. A prover must be able to check whether an extracted helper theorem still expresses the same obligation after being removed from its source proof, determine which helpers are independent and which depend on earlier discharged obligations, and replay a local helper theorem against the original proof step when that helper is reused.

We present SubgoalCalc, a certified decomposition layer for Lean 4. Its central object is a *certified decomposition*: a root goal, a finite family of residual obligations, an operational dependency relation among those obligations, and an assembly witness showing that proofs of the residual obligations reconstruct a proof of the root theorem. The decomposition is therefore represented as a proof-theoretic object inside Lean 4, with explicit obligations, dependencies, and reconstruction evidence.

We use the term *control plane* for the representation layer whose outputs are consumed by downstream proving components when they choose schedules, lift helper theorems, or reinject lemmas, while preserving replay links back to Lean 4. Certified decompositions support exact comparison and transformation through four operators: normalization, duplicate collapse, overlay, and refinement. Each operator rebuilds the assembly witness and the resulting object is checked again in Lean 4, giving a compositional algebra over local proof obligations.

The dependency relation carried by a decomposition gives the controller enough structure to select admissible schedules. Independent obligations can be exposed to parallel solving, while dependency-bearing obligations impose staged execution. In our downstream experiments, this dependency geometry changes the internal behavior of a real proving stack and determines which schedules satisfy the induced contract.

The same representation also supports replay-backed lemma reuse. Residual obligations can be lifted to closed theorems, minimized, replay-checked against the original local proof body, and packaged for later use. This turns local proof obligations into small helper libraries with explicit replay evidence, complementing offline theorem-mining approaches.

We evaluate SubgoalCalc across generated exact suites, downstream prover integrations, and corpus-scale extraction. The exact suites test the algebra on dependency-bearing DAGs and geometry-changing transforms. In an integration with HILBERT, certified extraction removes structural extraction failures on a curated stress slice. On DepBench, the controller satisfies a schedule contract induced by dependency geometry. Replay-backed helper lemmas compress proof plans, transfer across held-out HILBERT targets, and cross from HILBERT into DepBench. A second downstream study closes a small white-box Aesop

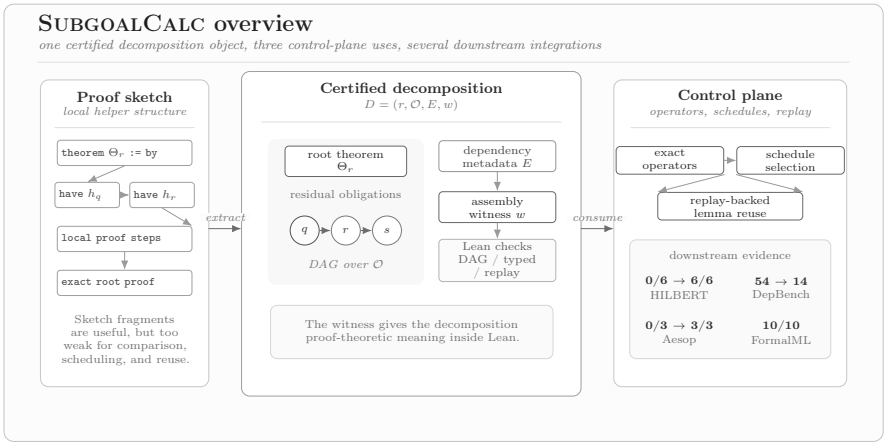

Figure 1: Overview of SubgoalCalc. A single certified decomposition object is used for exact transformation, schedule selection, replay-backed lemma extraction, and downstream integration.

feasibility slice. On FormalML, SubgoalCalc performs exact goalframe extraction on all 4,937 examples and localizes the remaining replay boundary through audited probes.

Overall, SubgoalCalc provides a certified representation of local proof decomposition in Lean 4, an exact algebra over that representation, a dependency-aware controller for schedule selection, a replay-backed lemma pipeline for proof-plan compression and transfer, and a corpus-scale account of exact extraction and replay opening on FormalML.

2 Certified decomposition

2.1 Goal frames and certified decompositions

A local helper theorem inside a Lean 4 proof carries more structure than its target formula. It has a local context, a dependency profile, and elaboration state. SubgoalCalc represents this information through a *goal frame*.

Definition 1 (Goal frame)**.** *A goal frame is a tuple*

$$g = (\Gamma, T, \Theta, \Delta, s),$$

where Γ is the ordered local context, T is the target type, Θ is the closed theorem type obtained by abstracting Γ over T, Δ records dependencies on earlier local helper theorems, and s records whether unresolved synthetic metavariables remain.

In the implementation, a goal frame is computed from a goal metavariable in Lean 4's metaprogramming layer. Expressions are sanitized before comparison:

assigned metavariables are instantiated, metadata and annotation wrappers are removed, and reducible definitions are weak-head normalized. This produces stable closed theorem types for equality tests, replay, and witness typing. Universe parameters and instance binders remain part of the elaborated theorem type. Unresolved synthetic metavariables cause certification rejection.

Definition 2 (Certified decomposition)**.** *Let r be a root goal frame. A certified decomposition of r is a tuple*

$$D = (r, O, E, w),$$

where $O = (o_1, \ldots, o_m)$ is an ordered family of residual obligations, $E \subseteq O \times O$ is an acyclic dependency relation, and

$$w : \Theta_{o_1} \to \cdots \to \Theta_{o_m} \to \Theta_r$$

is an assembly witness accepted by Lean *4.*

The witness gives the decomposition a direct proof-theoretic meaning: proofs of the residual obligations reconstruct a proof of the root theorem.

The witness and the dependency relation have different roles. The witness is checked directly by Lean 4. The dependency relation records dependencies exposed by elaborated helper bodies, is checked for acyclicity, and is used operationally through replay and schedule evaluation. In this paper, E is the extracted operational dependency structure associated with certification and replay.

Proposition 1 (Certification soundness)**.** *If $D = (r, O, E, w)$ is certified and each Θ_{o_i} is inhabited, then Θ_r is inhabited.*

Complete proofs appear in the appendix.

2.2 Exact operators

SubgoalCalc equips certified decompositions with four operators.

Normalization. Normalization canonicalizes obligation order and rewrites dependency references accordingly. It preserves certification and the root theorem.

Duplicate collapse. Duplicate collapse quotients obligations by equality of sanitized closed theorem type, redirects all incoming references to a representative obligation, and rebuilds the witness on the quotient. This quotient is sound at the proof-obligation layer. Source provenance is stored separately during replay-backed reuse, keeping theorem-key equality and proof origin as distinct pieces of evidence.

Overlay. Given two certified decompositions with the same root theorem, overlay forms their certified superposition. It unions the obligation families after identifier shifting and weakens the witness over the larger telescope. Overlay acts as a join in decomposition space. In this paper, it supports exact comparison and aggregation over certified decompositions.

Refinement. Refinement substitutes a certified child decomposition for one selected residual obligation in a parent decomposition. If the child root theorem matches that residual theorem, the refined witness is obtained by functional composition.

Proposition 2 (Certification preservation)**.** *Normalization, duplicate collapse, overlay, and refinement preserve certification.*

These operators act on certified objects whose outputs are re-checked by Lean 4. The implementation also computes a reduced *structural core* by canonicalization and quotienting, which is used for exact equality and algebraic counting in the experiments.

2.3 Schedules induced by dependency structure

A decomposition induces a scheduling problem. Independent residual obligations can be solved in parallel, while dependency-bearing obligations require staged execution.

Definition 3 (Admissible schedule)**.** *Let $D = (r, O, E, w)$ be a certified decomposition. An ordered partition $(B_0, \ldots, B_k)$ of O is admissible if for every edge $(o_i, o_j) \in E$, the batch containing o_i appears strictly before the batch containing o_j.*

Operationally, the schedules are *flat parallel*, which places all obligations in one batch, and *dependency staged*, which groups obligations by depth in the dependency DAG.

Theorem 1 (Schedule laws)**.** *For every certified decomposition: (i) the depth-layered schedule is admissible; and (ii) the flat-parallel schedule is admissible if and only if the dependency graph is edgeless.*

These laws turn dependency structure into a controller decision used in the experiments as both a schedule contract and a regression-test target for dependency-aware execution.

3 Control and replay-backed reuse

3.1 Kernel-faithful extraction from proof sketches

The preferred extraction route operates on elaborated proof structure. Given a proof sketch with local helper theorems, SubgoalCalc builds goal frames for

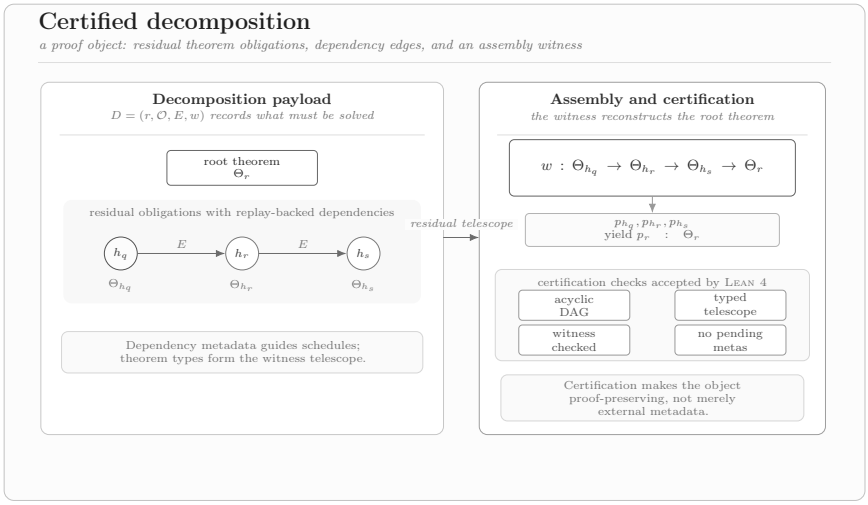

Figure 2: Certified decomposition as a proof object. Residual obligations carry dependency edges and an explicit assembly witness accepted by Lean 4.

the root and residual obligations, constructs the dependency DAG, and emits a certified decomposition whose witness is checked by Lean 4. This preserves local structure that textual extraction can drop, especially when helper theorems depend on earlier local steps.

3.2 Controller

The controller consumes a certified decomposition and selects a solving schedule. In the experiments, we use four modes: `off`, `flat_parallel`, `dependency_staged`, and `auto`. The `auto` mode applies the schedule laws from Section 2: edgeless decompositions are assigned to flat-parallel execution, and dependency-bearing decompositions are assigned to dependency-staged execution. The controller also accounts for helper-lemma reinjection, since successful reinjection can collapse dependency edges and move a target from the staged fragment to the flat fragment.

3.3 Replay-backed lemma lifting

Residual obligations are turned into reusable helper lemmas through lifting, replay, and reinjection.

Lifting. A residual obligation is lifted to a closed theorem statement by abstracting its local context and retaining the declarations needed for a well-typed theorem.

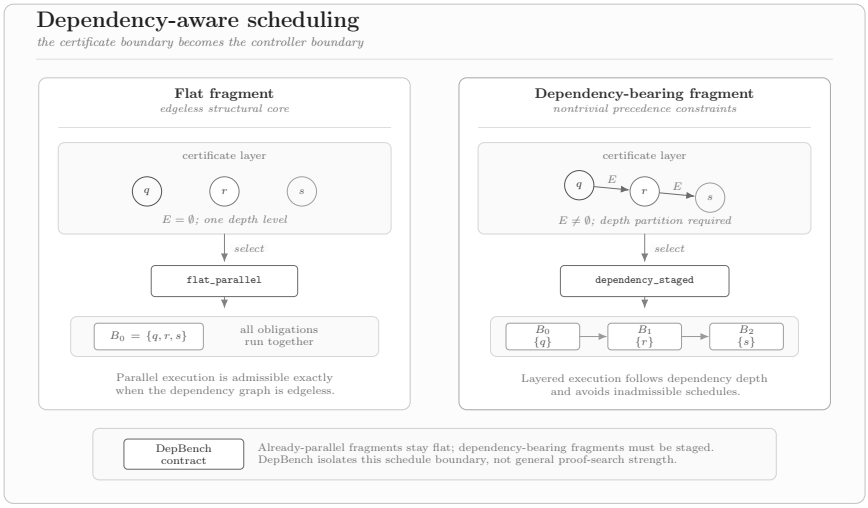

Figure 3: Dependency-aware schedule selection. The same schedule boundary appears in the certificate layer and in the controller's choice between parallel and layered execution.

Replay. The lifted statement is paired with the original local proof body, and the resulting theorem is replay-checked by Lean 4. Replay establishes that the lifted theorem still corresponds to the local proof step after abstraction.

Reinjection. Replay-checked helper lemmas are packaged into small libraries and reintroduced into later proof attempts. Reinjection is selective. A transferred lemma is useful when it matches a residual obligation in the target proof and changes the plan geometry in a favorable way, for example by eliminating a dependency-bearing helper and enabling a flat-parallel schedule.

The pipeline observes lemma reuse through downstream behavior: a helper lemma survives lifting and replay, is reintroduced into a later proof attempt, and is credited when its presence changes the selected plan or the proof search outcome.

4 Experimental design

The evaluation follows the lifecycle of a certified decomposition: construction and transformation, use by a downstream controller, and reuse through replay-checked helper lemmas.

Exact suites. Generated certified suites validate the implementation and the algebraic operators. The first suite contains 1,000 flat decompositions generated from four operator families. The second contains 750 certified dependency-

bearing DAGs across three families, with up to 28 obligations and dependency depth up to 27. A third suite contains 1,500 certified transforms that cross the schedule boundary in both directions.

HILBERT integration. We use a curated six-problem slice that stresses local helper-theorem extraction and dependency handling. The main comparison uses a textual extraction baseline. We also report a retry-enabled baseline.

DepBench. DepBench is a dependency-sensitive scheduling benchmark with six cases: two flat-parallel cases and four dependency-bearing cases. It isolates controller behavior while keeping the rest of the proving stack fixed, and serves as a constructed schedule-contract benchmark.

Replay-backed reuse. We study reuse under within-problem reinjection on the chain slice of the HILBERT benchmark, held-out transfer across six HILBERT targets, and cross-corpus transfer from HILBERT into DepBench and FormalML.

White-box augmentation. We evaluate a second downstream regime on a small Aesop slice with three flat-fragment fixtures. One fixture contains duplicate residual obligations and exposes quotient savings directly. This slice serves as a white-box feasibility check.

FormalML. We use the full FormalML train split of 4,937 examples to study corpus-scale extraction and replay. The corpus produces 3,041 residual obligations, 2,362 unique theorem keys, and 679 duplicate keys. For replay, we report exact extraction at scale and use stratified probes to separate provisioning, serializer normalization, and source-aligned family repair.

Metrics. The main metrics are certification rate, replay rate, dependency recall, controller contract satisfaction, admissibility, schedule cost, proof-plan cost under reinjection, transfer selectivity, and exact corpus coverage. Schedule and proof-plan costs are structural accounting scores, so we report their components alongside the totals.

5 Results

5.1 The exact algebra exposes a small stable basis and a dependency-bearing regime

The large flat suite reduces 1,000 certified decompositions to four structural cores. All 1,000 entries satisfy the schedule laws. Since the suite is edgeless, flat-parallel execution is admissible throughout. The dependency-bearing suite exposes a separate regime: 750 certified DAGs, 500 of them dependency-bearing,

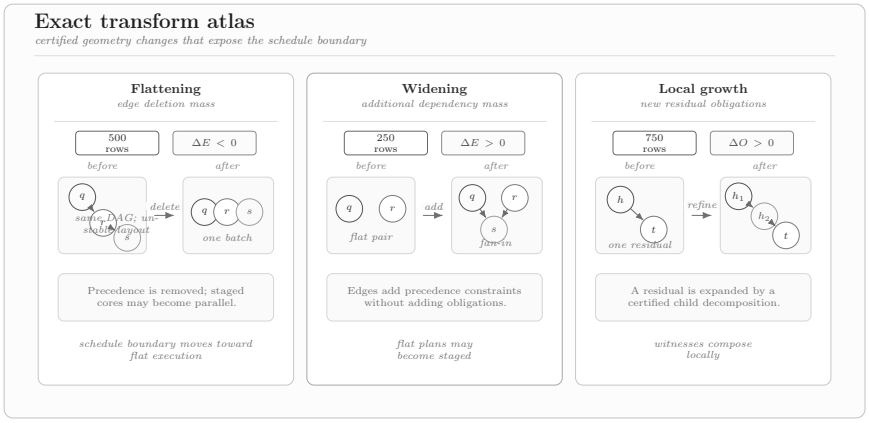

Figure 4: Exact transform atlas. The certified transform layer already exposes flattening, widening, local growth, and schedule-boundary crossings.

Table 1: HILBERT integration.

Metric	Textual baseline	Retry baseline	SubgoalCalc	Best delta
Final successes	0/6	4/6	6/6	+6, +2
Helper replay rate	7/12	—	12/12	+5
Dependent-helper replay	1/6	—	6/6	+5
Invalid subgoal checks	12	0	0	−12
Extraction-related reasoner calls	14	4	0	−14, −4
Dependency lift rate	0.000	0.417	0.417	+0.417

with 10,750 total dependency edges and depth up to 27. Across 1,500 certified transforms, 750 cases cross the schedule boundary. These suites provide large-scale regression coverage for exact-layer invariants.

The algebra preserves certification and exercises the same schedule boundary used later by the controller. The appendix reports the exact counts, while the main text focuses on the structural regimes exposed by the transform layer.

5.2 Kernel-faithful extraction removes structural failures in HILBERT

On the curated six-problem HILBERT stress slice, the textual extraction route compiles all six sketches but replays only 7 of 12 helper theorems and only 1 of the 6 dependency-bearing helpers. The certified route replays all 12 helpers and all 6 dependent ones, with perfect dependency-edge recovery.

The same difference appears in end-to-end behavior. Against the textual baseline, success rises from 0/6 to 6/6, invalid helper checks drop from 12 to 0, and extraction-related reasoner calls fall from 14 to 0. Against the retry-enabled baseline, success rises from 4/6 to 6/6 and extraction-related reasoner calls fall from 4 to 0.

Table 2: Dependency-aware controller on DepBench.

Metric	Baseline	SubgoalCalc controller	Delta
Contract-satisfying runs	2/6	6/6	+4/6
Gated dep.-bearing runs satisfied	0/4	4/4	+4/4
Dependency-staged runs	0	4	+4
Inadmissible schedule runs	4	0	−4
Total schedule cost	54	14	−40

5.3 Controller decisions become exact on dependency-bearing cases

On DepBench, the controller is the only changing component. The baseline keeps all four dependency-bearing cases in flat parallel, violating the induced schedule constraints. The certified controller switches exactly those four cases to dependency-staged execution, leaves the already-flat cases unchanged, and raises controller contract satisfaction from 2/6 to 6/6. This experiment isolates the schedule contract induced by dependency geometry.

Schedule-cost gains concentrate in the gated dependency-bearing slice, while the already-admissible parallel slice is unchanged. Generate-proof calls stay fixed, inadmissibility risk drops from 4 to 0, and helper reuse credit rises from 0 to 4. This matches the exact transform results: the controller intervenes on decompositions located on the dependency-bearing side of the schedule boundary.

5.4 Replay-backed helper lemmas compress proof plans and transfer selectively

Within-problem reinjection on the chain slice of the curated HILBERT benchmark collapses both chain cases from layered plans to one-batch plans and reduces proof-plan cost from 16 to 6. In held-out transfer, a four-entry replay-backed micro-library spanning two structural families produces 20 replay-checked placements across six distinct HILBERT targets: two targets flatten, four remain unchanged, none is harmed, and total proof-plan cost falls from 48 to 38.

The same library transfers selectively across corpora. In DepBench, one target flattens and flips from failure to success; on the audited FormalML fragments, exact overlap remains zero because the library replays but its theorem keys do not match the audited residual obligations. The appendix reports the detailed transfer table, while the main text uses these results to characterize replay-backed reuse as fragment-selective compression of dependency-bearing plans.

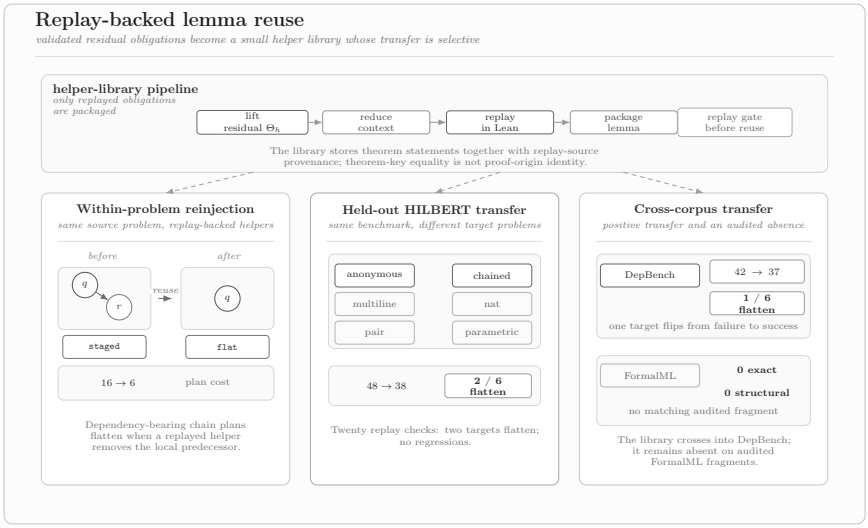

Figure 5: Replay-backed lemma reuse and transfer. It compresses dependency-bearing plans locally, transfers across held-out HILBERT targets, and crosses into DepBench while remaining absent on the audited FormalML fragments.

5.5 A second downstream regime closes on a white-box Aesop slice

The same representation supports a flat, white-box proof-search regime. On three Aesop fixtures, all in the flat fragment, local forward rules derived from the decomposition close the full slice from 0/3 to 3/3. One fixture contains duplicate residual obligations, and quotienting removes one redundant obligation before rule generation. This slice serves as a white-box feasibility check for rule generation from certified decompositions.

5.6 FormalML separates exact extraction from replay opening

At corpus scale, FormalML behaves differently from the dependency-sensitive slices. The extraction layer is exact on all 4,937 examples and all 3,041 residual obligations. The resulting residual graph is predominantly flat, so FormalML primarily evaluates exact extraction and replay opening.

Replay coverage depends on the available workspace roots and the normalization layer used during checking. In the main workspace, missing external project roots block replay; once those roots are provisioned at the pinned revisions, the replay boundary opens on all audited probes. On the stratified probes, raw exact replay succeeds on 6/10, 3/10, and 3/10 examples for the 100, 500, and full slices. Serializer normalization raises the two larger probes to 5/10. Source-aligned family repair closes the remaining residual rows, producing 10/10 final replay on

all three probes. These repair families are reported as a diagnostic replay-opening layer.

6 Related work

Recent work has established several roles for subgoals in formal reasoning. FormalML formulates subgoal completion as a benchmark task in Lean 4 and studies local obligation discharge under complex contexts and retrieval [Y]. Zhao et al. and SubgoalXL use subgoal-based proofs as supervision targets for in-context proving and expert learning in Isabelle [Z, Z]. HILBERT uses recursive decomposition to connect informal reasoning with formal proof construction, while DeepSeek-Prover-V2 uses decomposition to synthesize training data and stabilize reinforcement learning [V, R]. Bourbaki formalizes self-generated subgoals as dynamic control targets in goal-conditioned search [Z]. MathlibLemma studies theorem-library growth by mining missing folklore lemmas [L].

SubgoalCalc sits between extraction, search, and reuse by providing a certified decomposition object that records residual obligations, dependency geometry, and an assembly witness accepted by Lean 4. Existing decomposition-based systems use subgoals as task units, prompts, search states, or training artifacts; SubgoalCalc gives the decomposition itself replay semantics, algebraic operations, and controller-facing structure. The resulting object is close to proof planning [B] and white-box proof search in Aesop [L], with an implementation in modern Lean 4 and an interface aimed at current decomposition-based theorem-proving agents.

7 Conclusion and limitations

SubgoalCalc represents local proof decomposition as a certified object inside Lean 4, containing residual obligations, dependency geometry, and an assembly witness checked by the prover. This representation gives exact transformation, dependency-aware scheduling, and replay-backed helper-lemma reuse a common control plane.

Across the evaluated regimes, the same control plane supports algebraic validation, extraction, scheduling, proof-search integration, and corpus-scale replay analysis. The exact suites validate the algebra and exercise the schedule boundary at scale. The HILBERT integration shows that kernel-faithful extraction removes structural failures that remain under textual extraction. The DepBench study isolates schedule selection and shows that dependency geometry can drive exact controller decisions. The Aesop slice shows that the same representation can feed a second white-box proof-search regime. The FormalML study shows corpus-wide exact extraction and separates extraction from replay opening on audited probes.

The assembly witness is checked directly by Lean 4, while the dependency relation is operational metadata extracted from elaborated helper bodies and

validated through acyclicity, replay, and schedule behavior. The exact suites are generated validation batteries. The HILBERT, DepBench, and Aesop studies are small, purpose-built stress slices. The FormalML replay study reports stratified probe results, with source-aligned family repair used as a diagnostic replay-opening layer.

These boundaries point to the next technical target: expanding the reusable fragment while preserving replay fidelity. The results establish certified decomposition as a practical representation layer for theorem-proving agents, connecting local proof structure to exact algebra, controller decisions, and reusable helper libraries.

References

[B] Alan Bundy. Proof planning. In *Proceedings of the Third International Conference on Artificial Intelligence Planning Systems*, AIPS 1996, pages 261–267. AAAI Press, 1996. URL `https://cdn.aaai.org/AIPS/1996/AIPS96-033.pdf`.

[d] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In André Platzer and Geoff Sutcliffe, editors, *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635, Cham, 2021. Springer. doi: 10.1007/978-3-030-79876-5_37. URL `https://doi.org/10.1007/978-3-030-79876-5_37`.

[L] Jannis Limperg and Asta Halkjær From. Aesop: White-box best-first proof search for Lean. In *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs*, CPP 2023, pages 253–266, New York, NY, USA, 2023. Association for Computing Machinery. doi: 10.1145/3573105.3575671. URL `https://doi.org/10.1145/3573105.3575671`.

[L] Xinyu Liu, Zixuan Xie, Amir Moeini, Claire Chen, Shuze Daniel Liu, Yu Meng, Aidong Zhang, and Shangtong Zhang. MathlibLemma: Folklore lemma generation and benchmark for formal mathematics. *arXiv preprint arXiv:2602.02561*, 2026. doi: 10.48550/arXiv.2602.02561. URL `https://arxiv.org/abs/2602.02561`.

[R] Z. Z. Ren, Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanjia Zhao, Liyue Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, Z. F. Wu, Zhibin Gou, Shirong Ma, Hongxuan Tang, Yuxuan Liu, Wenjun Gao, Daya Guo, and Chong Ruan. DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv preprint arXiv:2504.21801*, 2025. doi: 10.48550/arXiv.2504.21801. URL `https://arxiv.org/abs/2504.21801`.

[V] Sumanth Varambally, Thomas Voice, Yanchao Sun, Zhifeng Chen, Rose Yu, and Ke Ye. Hilbert: Recursively building formal proofs with informal

reasoning. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=GN8OdkTo3B.

[Y] Xiao-Wen Yang, Zihao Zhang, Jianuo Cao, Zhi Zhou, Zenan Li, Lan-Zhe Guo, Yuan Yao, Taolue Chen, Yu-Feng Li, and Xiaoxing Ma. FormalML: A benchmark for evaluating formal subgoal completion in machine learning theory. *arXiv preprint arXiv:2510.02335*, 2025. doi: 10.48550/arXiv.2510.02335. URL https://arxiv.org/abs/2510.02335.

[Z] Xueliang Zhao, Wenda Li, and Lingpeng Kong. Subgoal-based demonstration learning for formal theorem proving. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 60832–60865. PMLR, 2024. URL https://proceedings.mlr.press/v235/zhao24h.html.

[Z] Xueliang Zhao, Lin Zheng, Haige Bo, Changran Hu, Urmish Thakker, and Lingpeng Kong. SubgoalXL: Subgoal-based expert learning for theorem proving. *arXiv preprint arXiv:2408.11172*, 2024. doi: 10.48550/arXiv.2408.11172. URL https://arxiv.org/abs/2408.11172.

[Z] Matthieu Zimmer, Xiaotong Ji, Rasul Tutunov, Anthony Bordg, Jun Wang, and Haitham Bou Ammar. Bourbaki: Self-generated and goal-conditioned MDPs for theorem proving. *arXiv preprint arXiv:2507.02726*, 2025. doi: 10.48550/arXiv.2507.02726. URL https://arxiv.org/abs/2507.02726.

A Formal properties

This appendix gives complete proofs of the structural results used in the main text.

Proposition 3 (Certification soundness)**.** *If $D = (r, O, E, w)$ is certified and each residual theorem Θ_{o_i} is inhabited, then the root theorem Θ_r is inhabited.*

Proof. For each residual obligation o_i, let $p_i : \Theta_{o_i}$ be a proof term. Since the witness is well typed,

$$w\, p_1 \cdots p_m$$

is a term of type Θ_r. Hence Θ_r is inhabited. □

Proposition 4 (Normalization preserves certification)**.** *If D is certified, then* Normalize(D) *is certified and has the same root theorem.*

Proof. Normalization only permutes the residual telescope and rewrites dependency references accordingly. Let the original telescope be

$$\Theta_{o_1} \to \cdots \to \Theta_{o_m} \to \Theta_r,$$

and let π be the permutation induced by normalization. The normalized witness is

$$w_{\text{norm}}(x_1, \ldots, x_m) = w(x_{\pi^{-1}(1)}, \ldots, x_{\pi^{-1}(m)}).$$

This term has the normalized telescope type, the codomain remains Θ_r, and the dependency graph stays acyclic because normalization relabels nodes but adds no edges. □

Proposition 5 (Duplicate collapse preserves certification)**.** *If D is certified, then* Quotient(D) *is certified and has no more residual obligations than D.*

Proof. Duplicate collapse groups obligations by equality of their sanitized closed theorem type and keeps one representative per class. All references to eliminated nodes are redirected to the representative. Since obligations in the same class have the same theorem statement, the witness can be rewritten by replacing each eliminated argument by the corresponding representative argument. Collapsing duplicate nodes in a DAG cannot create a nontrivial directed cycle; at worst it creates self-loops, which are removed. Therefore the quotient remains acyclic and certified, with weakly fewer obligations. □

Proposition 6 (Overlay preserves certification)**.** *Let D_1 and D_2 be certified decompositions with the same root theorem. Then* Overlay(D_1, D_2) *is certified.*

Proof. Overlay takes the disjoint union of the obligation families after identifier shifting and preserves the shared root theorem. Since no new cross-edges are introduced between the two input graphs, the output graph is acyclic. If

$$w_1 : \Theta_{a_1} \to \cdots \to \Theta_{a_m} \to \Theta_r$$

is the witness of D_1, then a valid witness for the overlaid telescope

$$\Theta_{a_1} \to \cdots \to \Theta_{a_m} \to \Theta_{b_1} \to \cdots \to \Theta_{b_n} \to \Theta_r$$

is obtained by weakening:

$$w_{\text{ov}}(x_1, \ldots, x_m, y_1, \ldots, y_n) = w_1(x_1, \ldots, x_m).$$

Thus the overlaid decomposition is certified. □

Proposition 7 (Refinement preserves certification)**.** *Let D_P be a certified parent decomposition and D_C a certified child decomposition whose root theorem matches one selected residual obligation of D_P. Then* Refine(D_P, D_C) *is certified and has the same root theorem as D_P.*

Proof. Let the selected parent residual obligation have theorem statement Θ_o. Because the child root theorem is also Θ_o, the child witness has type

$$w_C : \Theta_{c_1} \to \cdots \to \Theta_{c_k} \to \Theta_o.$$

If the parent witness has type

$$w_P : \Theta_{p_1} \to \cdots \to \Theta_o \to \cdots \to \Theta_{p_m} \to \Theta_r,$$

then the refined witness is the composition

$$w_{\mathrm{ref}}(\vec{x}, \vec{y}, \vec{z}) = w_P(\vec{x}, w_C(\vec{y}), \vec{z}).$$

The refined graph replaces one node by an acyclic child graph and redirects edges through the child root; acyclicity is preserved. The codomain remains Θ_r, so certification is preserved. □

Definition 4 (Depth)**.** *For a certified decomposition $D = (r, O, E, w)$ and obligation $o \in O$, the depth $d(o)$ is the length of the longest directed path ending at o.*

Theorem 2 (Depth-layered schedules are admissible)**.** *Let $B_t = \{o \in O : d(o) = t\}$. Then $(B_0, B_1, \ldots, B_k)$ is an admissible schedule.*

Proof. Take any edge $(o_i, o_j) \in E$. Any path ending at o_i can be extended by this edge to obtain a longer path ending at o_j. Therefore $d(o_i) < d(o_j)$, so o_i appears in an earlier batch than o_j. This is exactly admissibility. □

Corollary 1 (Flat parallel admissibility)**.** *The one-batch flat-parallel schedule is admissible if and only if the dependency graph is edgeless.*

Proof. If the graph has no edges, there are no precedence constraints and the one-batch schedule is admissible. If there exists an edge (o_i, o_j), admissibility requires o_i to appear strictly before o_j, which is impossible in a single batch. □

B Additional experimental detail

Certification scope. The Lean-checked assembly witness certifies root reconstruction from residual obligations. Dependency edges are replay-backed operational dependencies extracted from elaborated proof bodies and checked for acyclicity and replay consistency. We do not claim that the extracted DAG is a minimal semantic dependency basis.

Exact suites. The flat suite contains four certified operator families and 1,000 total rows. The dependency-bearing suite contains three families and 750 certified rows, including 250 chain and 250 mixed cases. The transform suite contains 1,500 certified transforms with 750 schedule-mode crossings.

HILBERT extraction study. The curated slice contains six problems. The exact route replays all 12 extracted helper theorems and all 6 dependency-bearing helpers. The textual route replays 7/12 helper theorems and 1/6 dependency-bearing helpers.

Controller benchmark. DepBench contains six cases: four dependency-bearing and two already-flat cases. Contract satisfaction measures whether the selected schedule agrees with the certified dependency geometry. Schedule cost is the benchmark's aggregate cost function over selected schedules.

Lemma reuse. The replay-backed micro-library used in the transfer experiments contains four entries spanning two structural families. Held-out transfer reports only off-diagonal placements. Cross-corpus transfer uses the same replay-checked library without additional tuning.

FormalML. On the full train split, 4,937 examples yield 3,041 residual obligations. For replay, we report three stratified probes of size 10, corresponding to the 100-example, 500-example, and full-slice audits. The replay progression separates exact extraction, provisioned statement replay, serializer normalization, and source-aligned family repair.

White-box augmentation. The Aesop slice contains three fixtures, all in the flat fragment. Rule augmentation uses local forward rules synthesized from replay-backed residual obligations.

Table 3: Lean feature handling.

Feature	Treatment	Boundary / failure mode
Universe parameters and instance binders	Preserved in the elaborated theorem type and replayed verbatim	No erasure-based quotienting across distinct elaborated binders
Local `let` declarations	Lifted through the reconstructed local context in declaration order	Fails if replay would leave unresolved metavariables
Reducible definitions	Sanitized by weak-head normalization at the reducible layer only	Not a full semantic normalization procedure
Synthetic metavariables	Rejected at certification / replay time if unresolved	Extraction is kernel-faithful only for accepted replayed terms
Duplicate theorem keys	Closed theorem-key equality is used for obligation quotienting	Replay-backed reuse keeps provenance and replay class separate

Table 4: Additional exact-suite statistics.

Suite	Entries	Max obligations	Max width	Max depth/stages	Edge total / delta
Flat certified suite	1,000	4	4	0	0
Dependency DAG suite	750	28	28	27	10,750
Certified transforms	1,500	29	—	26	−7,250 net

Table 5: HILBERT stress-slice taxonomy.

Problem	Textual baseline	Retry	Certified	Main structural issue
anon. chain	fail	success	success	anonymous helper lifting
chained	fail	fail	success	predecessor binder missing in helper replay
multiline dep.	fail	success	success	multiline dependency preservation
nat chain	fail	success	success	dependency-bearing local theorem promotion
pair chain	fail	success	success	pair-structured helper interface preservation
parametric	fail	fail	success	lifted parameter binder required for replay

Table 6: Cost components reported in the main text.

Benchmark	Setting	Risk	H	D	Calls	Overhead	Reuse / reinj.	Total
DepBench	controller off	4	—	—	14	0	0	54
DepBench	controller auto	0	—	—	14	4	4	14
HILBERT reinjection	without pack	—	4	2	6	2	2	16
HILBERT reinjection	with pack	—	2	0	4	0	0	6

Table 7: Replay-backed reinjection and transfer detail.

Setting	Tasks	Baseline cost	With library	Delta	Main effect
Chain slice (within-problem)	2	16	6	−10	both chain cases flatten
Held-out HILBERT transfer	6	48	38	−10	2 flattened targets, no regressions
Cross-corpus transfer to DepBench	6	42	37	−5	one failure-to-success flip
Cross-corpus transfer to FormalML	audited probes	0 overlap	0 overlap	—	fragment mismatch, not replay failure

Table 8: Replay progression on FormalML probes.

Slice	Probe size	Exact replay	Normalized replay	Family-repair gain	Final replay
FormalML-100	10	6	6	4	10
FormalML-500	10	3	5	5	10
FormalML-all	10	3	5	5	10

Table 9: Replay-opening layers on FormalML probes.

Layer	What is counted	100 / 500 / all probes	Interpretation
Exact replay	Raw replay under pinned provisioning	6 / 3 / 3	Baseline replay boundary on audited rows
Serializer normalization	Removes serializer noise such as `to_theorem` wrappers	6 / 5 / 5	Diagnostic normalization layer, not raw replay
Surface normalization	Joined-line and notation cleanup	0 / 0 / 0 additional	Negative control: residual failures are not superficial formatting only
Family repair	Source-aligned repair families for Bennett and ProximalGradient rows	4 / 5 / 5 additional	Diagnostic replay-opening layer, not held-out automation

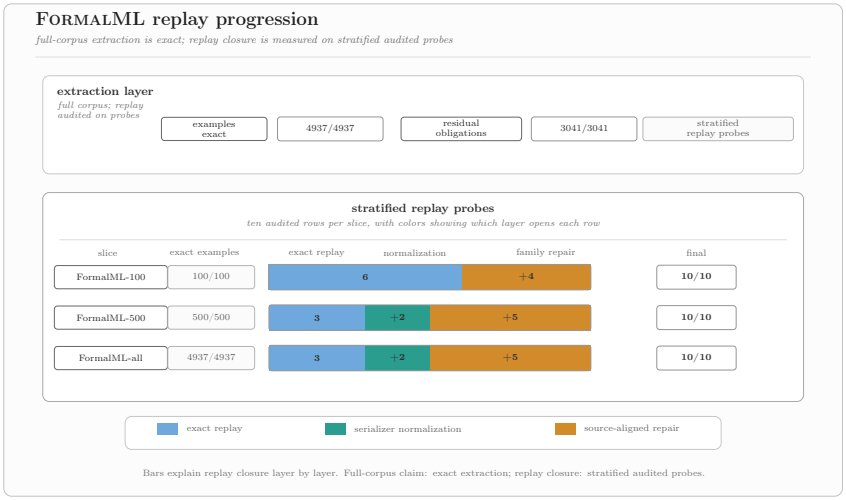

Figure 6: FormalML replay progression.

C Reproducibility and artifact summary

The supplementary material contains the Lean 4 implementation of SubgoalCalc, experiment runners, and curated benchmark slices for the downstream studies. The artifact is organized by run family: exact certified suites, downstream integrations, replay-backed reuse studies, and corpus-scale analyses.

The main external assets are Lean 4 4 [d], Aesop [L], HILBERT [V], FormalML [Y], and MathlibLemma [L]. The supplementary material records the versions used in the experiments and describes the scope of the anonymized review snapshot. For the FormalML replay study, the artifact separates exact

extraction, environment provisioning, serializer normalization, and source-aligned family repair, making the replay boundary auditable.

The reported results consist of deterministic structural counts, exact replay counts, controller-contract outcomes, and benchmark totals. For the curated downstream slices, the benchmark definitions are included in the submitted artifact. For the corpus-scale FormalML study, the artifact includes decomposition summaries, candidate counts, replay-progression tables, and the probe-level records used in the manuscript.

Compute resources. All reported runs are evaluation runs over certified decomposition operators, curated theorem-proving slices, and corpus-processing pipelines. The experiments require no training of large language models. The heaviest components are the corpus-scale FormalML extraction and replay studies and the downstream HILBERT integrations, all scripted in the supplementary material.

Asset licenses. The supplementary material credits the external projects and datasets used in the experiments and records their versions and applicable terms. The project-specific code and benchmark slices introduced by this work are documented in the anonymized review artifact.

D Broader impacts

This work improves the reliability and auditability of decomposition-based theorem proving. Certified decompositions expose residual obligations, dependency structure, and reconstruction evidence in a form that can be checked, transformed, scheduled, replayed, and reused inside Lean 4. These properties support formal mathematics, proof engineering, and verification workflows for software and hardware systems that depend on machine-checkable proof artifacts.

The same capabilities can strengthen formal reasoning pipelines used in high-assurance systems across many domains. The artifact is a research system centered on certified proof objects, replay checks, and explicit audit trails. Its main safety property is inspectability: decompositions, lifted helper lemmas, schedule decisions, and replay outcomes remain tied to Lean 4-checked evidence.